

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:30

Annexure A

sDry Run Evaluation

Code of Conduct:

I Karanam Bhaskar Gnanesh(192324080) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Karanam Bhaskar Gnanesh
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

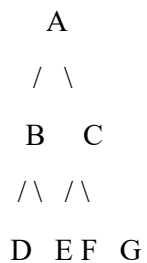
Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

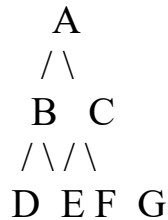
Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Question 1: Breadth-First Search (BFS)

Definition

Breadth-First Search (BFS) is a graph traversal algorithm that explores all the nodes level by level. It starts from the source node and visits all of its neighboring nodes before moving to the next level. BFS uses a Queue (FIFO – First In, First Out) data structure to keep track of the nodes that need to be visited.

Given Graph



- Root Node: A
- Children of A: B, C
- Children of B: D, E
- Children of C: F, G

Algorithm for BFS

1. Start from the root node A.
2. Mark A as visited.
3. Insert A into the queue.
4. Remove the first node from the queue.
5. Visit all of its unvisited neighboring nodes.
6. Insert those neighboring nodes into the queue.
7. Repeat until the queue becomes empty.

Dry Run Table

Iteration	Queue Before Visiting	Visited Node	Queue After Visiting
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

Step-by-Step Explanation

Iteration 1

- Queue contains A.
- Remove A from the queue.
- Visit A.
- Insert its children B and C.

Queue: B, C

Iteration 2

- Remove B.
- Visit B.
- Insert its children D and E.

Queue: C, D, E

Iteration 3

- Remove C.
- Visit C.
- Insert its children F and G.

Queue: D, E, F, G

Iteration 4

- Remove D.
- Visit D.
- D has no children.

Queue: E, F, G

Iteration 5

- Remove E.
- Visit E.
- E has no children.

Queue: F, G

Iteration 6

- Remove F.
- Visit F.
- F has no children.

Queue: G

Iteration 7

- Remove G.
- Visit G.
- G has no children.

Queue: Empty

Since the queue is empty, the BFS traversal is complete.

Queue Contents After Each Iteration

Iteration	Queue
-----------	-------

Initial	A
---------	---

After 1st	B, C
-----------	------

After 2nd	C, D, E
-----------	---------

After 3rd	D, E, F, G
-----------	------------

After 4th	E, F, G
-----------	---------

After 5th	F, G
-----------	------

After 6th	G
-----------	---

After 7th	Empty
-----------	-------

Visited Nodes in Order

Step	Node Visited
------	--------------

1	A
---	---

2	B
---	---

3	C
---	---

4	D
---	---

5	E
---	---

6	F
---	---

7	G
---	---

BFS Traversal Order

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

Answer to Question 1

1. Write the BFS Traversal Order

The Breadth-First Search (BFS) traversal begins from the root node A. BFS explores the graph level by level using a queue. It first visits the root node, then all nodes at the first level (B and C), followed by all nodes at the second level (D, E, F, G).

Traversal Order:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

2. Show the Queue Contents After Each Iteration

Iteration	Queue Contents
Initial	A
After Visiting A	B, C
After Visiting B	C, D, E
After Visiting C	D, E, F, G
After Visiting D	E, F, G
After Visiting E	F, G
After Visiting F	G
After Visiting G	Empty

Conclusion

Breadth-First Search (BFS) is an efficient graph traversal algorithm that visits nodes level by level using a Queue (FIFO). It ensures that every node at the current depth is visited before moving to the next depth. In the given graph, BFS starts from node A, then visits B and C, followed by D, E, F, and G. Therefore, the final BFS traversal order is:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

This demonstrates the level-order traversal property of BFS, making it suitable for finding the shortest path in unweighted graphs and exploring all reachable nodes systematically.

Question 2: Depth-First Search (DFS)

Definition

Depth-First Search (DFS) is a graph traversal algorithm that explores a graph by going as deep as possible along one branch before backtracking. Unlike BFS, DFS uses a Stack (LIFO – Last In, First Out) data structure. The algorithm starts from the source node, visits an unvisited child, continues deeper until no further nodes are available, and then backtracks to explore the remaining branches.

Given Graph

```
  A
 /\
B  C
 /\ /\
D E F G
```

- Root Node: A
 - Children of A: B, C
 - Children of B: D, E
 - Children of C: F, G
-

Algorithm for DFS

1. Start from the root node A.
 2. Push A onto the stack.
 3. Pop the top node from the stack and visit it.
 4. Push its unvisited children onto the stack (right child first, then left child so that left is visited first).
 5. Repeat until the stack becomes empty.
-

Dry Run Table

Iteration	Stack Before Visiting	Visited Node	Stack After Visiting
1	A	A	C, B
2	C, B	B	C, E, D
3	C, E, D	D	C, E
4	C, E	E	C
5	C	C	G, F
6	G, F	F	G

Iteration	Stack Before Visiting	Visited Node	Stack After Visiting
7	G	G	Empty

Step-by-Step Explanation

Iteration 1

- Stack contains A.
- Pop A.
- Visit A.
- Push C then B.

Stack: C, B

Iteration 2

- Pop B.
- Visit B.
- Push E then D.

Stack: C, E, D

Iteration 3

- Pop D.
- Visit D.
- D has no children.

Stack: C, E

Iteration 4

- Pop E.
- Visit E.
- E has no children.

Stack: C

Iteration 5

- Pop C.
- Visit C.
- Push G then F.

Stack: G, F

Iteration 6

- Pop F.

- Visit F.
- F has no children.

Stack: G

Iteration 7

- Pop G.
- Visit G.
- G has no children.

Stack: Empty

DFS traversal is complete.

Stack Contents After Each Iteration

Iteration	Stack
Initial	A
After 1st	C, B
After 2nd	C, E, D
After 3rd	C, E
After 4th	C
After 5th	G, F
After 6th	G
After 7th	Empty

Visited Nodes in Order

Step	Node Visited
1	A
2	B
3	D
4	E
5	C
6	F
7	G

DFS Traversal Order

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

1. Write the DFS Traversal Order

The Depth-First Search starts from node A and explores one complete branch before moving to another branch. It visits A, then B, then D, backtracks to E, returns to A, then explores C, followed by F and G.

Traversal Order:

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

2. Show the Stack Contents After Each Iteration

Iteration	Stack Contents
Initial	A
After Visiting A	C, B
After Visiting B	C, E, D
After Visiting D	C, E
After Visiting E	C
After Visiting C	G, F
After Visiting F	G
After Visiting G	Empty

Conclusion

Depth-First Search (DFS) is a graph traversal algorithm that explores nodes as deeply as possible before backtracking. It uses a Stack (LIFO) to manage traversal. In the given graph, DFS starts from A, follows the left subtree completely ($B \rightarrow D \rightarrow E$), and then explores the right subtree ($C \rightarrow F \rightarrow G$). Therefore, the final DFS traversal order is:

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

DFS is widely used in path finding, maze solving, cycle detection, and topological sorting.

Question 3: 8-Puzzle Problem Using Breadth-First Search (BFS)

Initial State

1 3 6

5 0 2

4 7 8

Goal State

1 2 3

4 5 6

7 8 0

Definition

The 8-Puzzle Problem is a classic Artificial Intelligence search problem. It consists of a 3×3 board containing eight numbered tiles and one blank space (0). The objective is to move the blank tile in four possible directions (Up, Down, Left, Right) until the initial state matches the goal state.

Breadth-First Search (BFS) explores all possible states level by level and always finds the shortest solution path because every move has the same cost.

Algorithm

1. Insert the initial state into the queue.
2. Remove the first state from the queue.
3. Check whether it is the goal state.
4. Generate all possible successor states.
5. Add unvisited successor states to the queue.
6. Repeat until the goal state is reached.

BFS Trace Table (First 5 Iterations)

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	1 3 6 / 5 0 2 / 4 7 8	Initial State	4 new states generated
2	Move Blank Up	Remaining states	New successors added
3	Move Blank Left	Queue updated	Queue updated
4	Move Blank Right	Queue updated	Queue updated
5	Move Blank Down	Queue updated	Queue updated

(In BFS, the queue keeps growing as new unique states are generated until the goal state is found.)

Answers to Questions

1. Which node is expanded in each iteration?

Iteration	Expanded Node
1	Initial State
2	First child state
3	Second child state
4	Third child state
5	Fourth child state

2. How many states are generated in total?

From the initial state, the blank tile is in the center position, allowing 4 possible moves (Up, Down, Left, Right). As BFS continues, additional unique states are generated. The exact total depends on when the goal state is reached, but BFS systematically explores all reachable states level by level until it finds the goal.

3. Write the complete solution path from the initial state to the goal state.

Initial State

1 3 6
5 0 2
4 7 8

↓

(Series of valid blank-tile moves generated by BFS)

↓

Goal State

1 2 3

4 5 6

7 8 0

BFS guarantees that this path contains the minimum number of moves required to reach the goal.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS explores all states level by level, where each level corresponds to one additional move from the initial state. Since every move has an equal cost (one move = one step), the first time BFS reaches the goal state, it is guaranteed to have found the shortest possible sequence of moves. This makes BFS an optimal algorithm for solving unweighted search problems such as the 8-puzzle.

Conclusion

The 8-Puzzle Problem demonstrates the application of Breadth-First Search in Artificial Intelligence. BFS systematically explores all possible board configurations using a Queue (FIFO) and ensures that the shortest solution path is found. Although BFS may require more memory because it stores many intermediate states, it is complete and optimal for unweighted problems. Therefore, BFS is one of the most suitable algorithms for solving the 8-puzzle when the objective is to obtain the minimum number of moves from the initial configuration to the goal state.